МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №4

«Професійне ООП та система типів Python»

з дисципліни «Професійний Python»

Варіант №5 — «Каталог автомобілів»

Виконав:
студент групи ФЕП-32
Горин Михайло
Репозиторій: github.com/MykhailoG212/Prof_python

Львів — 2026

2. Номер і тема лабораторної роботи

Лабораторна робота №4. Тема: «Професійне ООП та система типів Python». Звіт оформлено за структурою попередніх робіт із титульною сторінкою, постановкою задачі, описом реалізації, таблицями, лістингами, експериментальною частиною та висновком.

3. Мета роботи

Сформувати практичні навички побудови професійної типізованої об'єктної моделі: застосовувати класи, dataclass, інкапсуляцію, властивості, наслідування та композицію; описувати контракти через ABC і Protocol; використовувати generic class, TypeVar і TypedDict; застосовувати dependency injection та принципи SOLID; перевіряти модулі статичним аналізатором mypy.

4. Номер індивідуального варіанта

Варіант №5 — «Каталог автомобілів». Робота продовжує єдиний проєкт із ЛР1–ЛР3, використовує наявний пакет car_catalog і зберігає попередні функції та streaming pipeline.

5. Постановка задачі

Необхідно побудувати типізовану об'єктну модель каталогу транспортних засобів. Модель має представляти Vehicle, Car, ElectricCar, VehicleCatalog і VehicleSpecification; підтримати додавання автомобілів, пошук за маркою/моделлю, фільтрацію за роком, визначення найдорожчого авто, середньої ціни та мінімального пробігу. Зовнішнє сховище й критерії пошуку мають задаватися через abstractions.

5.1. Вимоги варіанта №5

Вимога	Реалізація

Вимога	Реалізація
Domain objects Vehicle, Car, ElectricCar	models.py: базовий Vehicle, Car та підтип ElectricCar
VehicleCatalog і VehicleSpecification	repositories.py: колекція каталогу; models.py: immutable specification
Додавання автомобіля	VehicleCatalog.add(); VehicleCatalogService.add()
Пошук за маркою / моделлю	BrandModelCriteria та VehicleCatalogService.search_by_brand_or_model()
Фільтрація за роком	YearCriteria та VehicleCatalogService.filter_by_year()
Найдорожче авто	VehicleCatalogService.most_expensive()
Середня ціна та найменший пробіг	VehicleCatalogService.average_price(), lowest_mileage()
Repository та search criteria abstractions	Repository[T] ABC, VehicleRepository і SearchCriteria[T] Protocol

5.2. Загальні вимоги до програми

Вимога ЛР4	Реалізація у проєкті
Не менше п’яти domain classes	Vehicle, Car, ElectricCar, VehicleSpecification, VehicleCatalog, VehicleCatalogService
Щонайменше два dataclass та frozen value object	Усі domain records — dataclass; VehicleSpecification frozen
Property та encapsulation	Vehicle.full_name; private _items та _repository; перевірки у __post_init__
Inheritance і composition	ElectricCar extends Car; VehicleCatalog has-a Repository[Vehicle]
ABC та Protocol	Repository[T] — ABC; VehicleRepository і SearchCriteria[T] — Protocol
Щонайменше три dunder methods	Vehicle.__str__; VehicleCatalog.__len__, __iter__, __contains__, __getitem__
Dependency injection та service layer	VehicleCatalogService отримує VehicleRepository у конструкторі
TypeVar та generic class	TypeVar T і Repository[T]/InMemoryRepository[T]
TypedDict	VehicleData у dto.py — типізований транспортний словник
Mypy / Pyright	mypy strict mode, налаштування у pyproject.toml
Демонстраційний сценарій у main.py	Пункт 13 запускає run_lab4_demo(); окремо python -m car_catalog.oop_demo

6. Структура проєкту

ООП-компоненти додані до наявного src-layout. Модулі розділено за відповідальністю: domain model, criteria, interfaces, repository, DTO, application service та composition root.

```
src/car_catalog/
├── models.py          # Vehicle, Car, ElectricCar, VehicleSpecification
├── criteria.py        # BrandModelCriteria, YearCriteria
├── protocols.py       # SearchCriteria[T], VehicleRepository
├── repositories.py    # Repository[T], InMemoryRepository[T], VehicleCatalog
├── dto.py             # VehicleData (TypedDict)
├── application.py     # VehicleCatalogService
├── oop_demo.py        # composition root і демонстраційний сценарій
└── main.py            # меню попередніх лабораторних, пункт 13 — ЛР4
```

Поточні модулі models.py, services.py та main.py продовжують використовуватися попередніми лабораторними. Модель Car зберегла сумісний набір обов'язкових аргументів brand, model, year, price, mileage; specification має коректне незмінне значення за замовчуванням.

7. Domain model та class diagram

Vehicle є спільною базовою сутністю з валідацією стану. Car наслідує базові поля; ElectricCar додає запас ходу і перевіряє тип пального. VehicleSpecification зберігає характеристики композиційно та є frozen value object.

```
Vehicle (ABC)
  └── Car
        └── ElectricCar


Vehicle *── VehicleSpecification       (* composition)
VehicleCatalog o── Repository[Vehicle]   (o aggregation / has-a)
InMemoryRepository[T] ──|> Repository[T] (implementation)
VehicleCatalogService ──> VehicleRepository (Protocol / DI)
VehicleCatalogService ──> SearchCriteria[Vehicle] (Protocol)
BrandModelCriteria, YearCriteria ──> SearchCriteria[Vehicle] (structural match)
```

Тип / клас	Роль та зв'язки
Vehicle	Базова dataclass-сутність; спільна валідація та full_name property.
Car	Конкретний тип транспортного засобу з типовою VehicleSpecification.
ElectricCar	Підтип Car; додає range_km і обмеження електричного типу пального.
VehicleSpecification	Незмінний value object: fuel_type, transmission, body_style.
VehicleCatalog	Колекція з композиційною залежністю від Repository[Vehicle].
VehicleCatalogService	Application service для пошуку та агрегатів; отримує repository через DI.

Тип / клас	Роль та зв'язки
BrandModelCriteria / YearCriteria	Незмінні стратегії пошуку зі спільним методом matches().

Наслідування відображає відношення is-a: ElectricCar є Car і успадковує валідовані атрибути. Композиція відображає has-a: каталог використовує сховище, але не є його різновидом. Такий зв'язок дозволяє замінити сховище без зміни моделі каталогу.

8. Dataclass, value object та encapsulation

Vehicle, Car, ElectricCar, VehicleSpecification, BrandModelCriteria і YearCriteria оголошені dataclass. Транспортні записи та технічна специфікація є frozen=True і slots=True: це відповідає value/data-oriented ролі цих об'єктів та не дозволяє змінити поля після конструювання. VehicleSpecification перевіряє непорожні значення у __post_init__.

Інкапсуляція підтримує валідний стан: Vehicle перевіряє марку, модель, рік, скінченну ціну та пробіг; ElectricCar викликає базову перевірку через super(), а потім перевіряє запас ходу й тип пального. Репозиторій приховує словник _items, а каталог надає контрольовані операції додавання та читання.

Property full_name обчислює представлення марки й моделі без дублювання збереженого поля. __str__ дає зручний для користувача опис транспортного засобу.

9. Abstract Base Class та Protocol

Repository[T] успадковує ABC та визначає abstractmethod add(), get() і all(). Не можна створити неповне concrete repository, доки воно не реалізує ці операції. InMemoryRepository[T] є concrete реалізацією generic контракту.

SearchCriteria[T] — runtime-checkable Protocol із методом matches(item). Конкретні BrandModelCriteria та YearCriteria не наслідують Protocol явно: вони структурно сумісні, бо надають потрібний метод. VehicleRepository окремо описує лише операції, потрібні application service.

ABC	**Protocol**
Номінальний контракт: реалізація успадковує Repository[T].	Структурний контракт: достатньо мати сумісні add/get/all або matches.
Підходить для гарантії повноти repository implementation.	Підходить для DI і підміни залежностей без жорсткого зв'язку.
Repository[T] має конкретні abstractmethod.	VehicleCatalogService залежить від VehicleRepository Protocol.

10. Generic repository, TypeVar та TypedDict

TypeVar T задає параметр типу для Repository[T] та InMemoryRepository[T]. Тип item під час add(), get() і all() зберігається узгодженим: InMemoryRepository[Car] повертає Car, а сховище іншого типу може зберігати інший domain object.

VehicleData у dto.py — TypedDict із полями id, brand, model, year, price, mileage та kind. Runtime-об'єкт лишається словником; сервіс використовує цей контракт для зовнішнього JSON-подібного представлення. Dataclass використовується для domain object із поведінкою та валідацією.

```
T = TypeVar("T")


class Repository(ABC, Generic[T]):
    @abstractmethod
    def add(self, item_id: int, item: T) -> None: ...


    @abstractmethod
    def get(self, item_id: int) -> T | None: ...


    @abstractmethod
    def all(self) -> Iterable[T]: ...
```

11. Application service, dependency injection і SOLID

VehicleCatalogService містить прикладний сценарій і отримує VehicleRepository через конструктор. Створення concrete сховища зосереджено в oop_demo.run_demo(), яке є composition root. Це dependency injection: залежність передається ззовні, а не створюється сервісом самостійно.

Принцип	Застосування
SRP — Single Responsibility	Repository зберігає записи; criteria визначають умови пошуку; service виконує use cases; DTO відповідає за зовнішнє представлення.
OCP — Open/Closed	Новий критерій додається новим класом із matches(); сервіс не потрібно розширювати гілками за конкретним типом.
LSP — Liskov Substitution	ElectricCar можна передавати всюди, де очікується Car або Vehicle; успадковані обмеження валідації зберігаються через super().
ISP — Interface Segregation	VehicleRepository та SearchCriteria — невеликі окремі контракти замість універсального інтерфейсу з непотрібними методами.
DIP — Dependency Inversion	High-level VehicleCatalogService залежить від VehicleRepository Protocol, а не від InMemoryRepository.

За варіанту A, коли сервіс сам створює InMemoryRepository, замінити його складніше, а тест із fake repository потребує зміни сервісу. За варіанту B залежність передається через конструктор: реалізацію легко замінити, окремо перевірити та розвивати.

12. Dunder methods та інтерфейс каталогу

Метод	Призначення
Vehicle.__str__()	Людиночитний рядок з назвою, роком і ціною.
VehicleCatalog.__len__()	Повертає кількість записів для len(catalog).
VehicleCatalog.__iter__()	Дозволяє перебирати транспортні засоби через for.
VehicleCatalog.__contains__()	Перевіряє наявність ID через конструкцію id in catalog.
VehicleCatalog.__getitem__()	Дозволяє отримати запис за ID через catalog[id]; відсутній ID дає KeyError.

Ці методи інтегрують доменну колекцію з природним Python-синтаксисом. __iter__ має явний Iterator[Vehicle], тому публічний API та очікуваний тип елементів зрозумілі статичному аналізатору.

13. Пошук та агрегування каталогу

Операція	Реалізація та поведінка
Пошук за маркою / моделлю	casefold() робить порівняння нечутливим до регістру; можна передати марку, модель або обидві.
Фільтр за роком	YearCriteria.matches() перевіряє точний рік випуску.
Найдорожче авто	max() із ключем price; порожній каталог повертає None.
Середня ціна	Один прохід за наявними записами; порожній каталог повертає 0.0.
Мінімальний пробіг	min() із ключем mileage; порожній каталог повертає None.

Фільтрування та пошук потребують O(n) часу й O(k) пам'яті для списку збігів, де k — кількість результатів. max/min та середня ціна потребують O(n) часу; додаткові дані обмежені поточним набором ітерації. Доступ за ID у внутрішньому словнику в середньому O(1). Сортування для звичайного пошуку не потрібне.

14. Демонстраційний сценарій та результат запуску

Команда python -m car_catalog.oop_demo створює InMemoryRepository[Vehicle], передає його VehicleCatalogService, додає два автомобілі з ДВЗ та один електромобіль, виконує пошук і показує агрегати. У головному меню той самий сценарій доступний у пункті 13.

```
--- ООП-модель каталогу (ЛР4, варіант 5) ---
Пошук Toyota: ['Toyota Camry (2020) — $25,000.00']
Авто 2023 року: ['Tesla Model 3 (2023) — $41,000.00']
Найдорожче авто: BMW M3 (2022) — $72,000.00
Середня ціна: $46,000.00
Мінімальний пробіг: Tesla Model 3 (2023) — $41,000.00
```

Результат узгоджується з демонстраційними даними: середня ціна дорівнює (25 000 + 72 000 + 41 000) / 3 = 46 000 USD; BMW M3 є найдорожчим, а Tesla Model 3 має найменший пробіг 12 000 км.

15. Експеримент зі статичною типізацією

Для статичного аналізу використано mypy 2.4.0 у strict mode. Залежність оголошена як optional group dev у pyproject.toml; конфігурація обмежує цільові файли сімома новими ООП-модулями. Додано py.typed, щоб пакет декларував наявність типових анотацій.

```
python -m pip install -e ".[dev]"
python -m mypy
Success: no issues found in 7 source files
```

15.1. Навмисні помилки та diagnostics

```
customer_id: int = "1"


def total() -> int:
    return "100"


repository: InMemoryRepository[Car] = InMemoryRepository()
repository.add(1, "not a car")
```

Для контрольного експерименту ці три помилки були запущені через mypy у тимчасовому файлі; тимчасовий файл після вимірювання видалено. Отримані diagnostics:

```
Incompatible types in assignment (expression has type "str", variable has type "int") [assignment]
Incompatible return value type (got "str", expected "int") [return-value]
Argument 2 to "add" of "InMemoryRepository" has incompatible type "str"; expected "Car" [arg-type]
Found 3 errors in 1 file
```

Виправлення: присвоїти customer_id ціле число 1; повертати 100 типу int; передати repository.add() об’єкт Car. Після виправлення робочі ООП-модулі проходять mypy без diagnostics. Неправильний експериментальний код не включено до src.

16. Дослідження наслідування та композиції

Підхід	У каталозі	Вплив
Наслідування	ElectricCar успадковує Car	Повторно використовує контракт авто й додає лише запас ходу; семантично коректне is-a.
Композиція	VehicleCatalog використовує Repository[Vehicle]	Репозиторій можна замінити; компоненти тестуються й розвиваються окремо.
Залежність від concrete class	Сервіс сам конструює InMemoryRepository	Сильніше зчеплення; складніше замінити storage або підставити fake.

Підхід	У каталозі	Вплив
DI через abstraction	Сервіс отримує VehicleRepository Protocol	Сховище передається ззовні; тестованість і розширюваність кращі.

17. Відповіді на контрольні запитання

Що таке class та object? Class описує спільну структуру й поведінку; object є конкретним екземпляром цього класу.

Що таке instance attribute? Поле, значення якого належить конкретному екземпляру; наприклад, brand автомобіля.

Що таке encapsulation? Контроль стану й допустимих змін через API об'єкта; Vehicle перевіряє поля під час створення.

Для чого property? Дає атрибутоподібний інтерфейс до контрольованого або обчислюваного значення; full_name складає марку й модель.

Що означає is-a? Підтип є різновидом базового типу; ElectricCar is a Car.

Що означає has-a? Об'єкт містить інший об'єкт як компонент; VehicleCatalog has a Repository.

Коли композиція краща за inheritance? Коли компонент можна незалежно замінювати чи комбінувати, а відношення is-a немає.

Для чого super()? Делегує спільну поведінку батьківській реалізації; ElectricCar викликає базову валідацію.

Що таке polymorphism? Одна операція працює з різними типами через спільний контракт, наприклад search із різними criteria.

Що таке ABC? Абстрактний базовий клас, який задає контракт через abstractmethod і не дозволяє створити неповну реалізацію.

Що таке Protocol? Опис структурного інтерфейсу: тип сумісний, якщо має потрібні методи та типи.

Чим ABC відрізняється від Protocol? ABC зазвичай вимагає номінального успадкування; Protocol перевіряє структуру методів.

Що таке dataclass? Декоратор, який автоматично генерує типові методи для класу даних, зокрема __init__ та __repr__.

Навіщо frozen=True? Забороняє повторне присвоєння полям після ініціалізації; зручно для immutable value object.

Для чого __post_init__()? Виконує додаткову валідацію після створення полів dataclass.

Що таке dunder methods? Спеціальні методи з подвійними підкресленнями, якими тип взаємодіє з Python protocol.

Для чого __iter__ та __len__? Вони дозволяють ітерувати каталог і отримати його розмір через len().

Для чого __contains__? Визначає перевірку належності через оператор in; каталог перевіряє наявність ID.

Що означає SRP? Клас має одну основну причину для зміни; persistence, search criteria та application use cases розділені.

Що означає OCP? Компонент відкритий до розширення і не потребує правок для кожної нової реалізації; додаємо новий criterion.

Що означає LSP? Підтип має замінювати базовий тип без порушення його контракту.

Що означає ISP? Краще кілька малих інтерфейсів, ніж один великий контракт із непотрібними методами.

Що означає DIP? Високорівнева логіка залежить від abstraction; сервіс залежить від VehicleRepository Protocol.

Що таке dependency injection? Передавання залежності ззовні, зазвичай через конструктор, замість створення її сервісом.

Що таке type hint? Анотація, яка документує типи й допомагає IDE та статичному аналізатору.

Чи перевіряє Python типи автоматично? Звичайний Python не виконує статичну перевірку анотацій; її робить окремий checker.

Що таке mypy? Статичний аналізатор, який зіставляє анотації та використання типів у програмі.

Для чого TypeVar? Позначає параметричний тип і зберігає зв'язок типів між аргументами та результатом.

Що означає Generic[T]? Оголошує параметризований клас, наприклад Repository[Car].

Що таке TypedDict? Опис словника з фіксованими ключами й типами значень; runtime-об'єкт лишається dict.

Чим TypedDict відрізняється від dataclass? TypedDict задає форму mapping payload; dataclass створює екземпляр класу, придатний для domain behavior.

Чому потрібне розділення DTO та domain object? Зовнішній формат даних може змінюватися незалежно від внутрішньої моделі предметної області.

Що перевіряє type-checking experiment? Чи знаходить checker несумісне присвоєння, return type та тип аргументу generic repository.

Який результат муру у проєкті? Після виправлень: Success: no issues found in 7 source files.

Чому service отримує repository через Protocol? Так можна замінити спосіб зберігання або підставити fake без зміни application logic.

18. Основні лістинги програми

Далі наведено файли, що реалізують предметну область ЛР4. Вони скопійовані з поточного проєкту після статичної перевірки; супровідні коментарі й docstrings збережені.

models.py

Лістинг наведено з робочого файлу src/car_catalog/models.py; форматування спрощено для звіту.

```
"""Типізована об'єктна модель каталогу автомобілів (ЛР4)."""


import math
from abc import ABC
from dataclasses import dataclass



@dataclass(slots=True, frozen=True)
class VehicleSpecification:
    """Незмінні технічні характеристики транспортного засобу."""


    fuel_type: str
    transmission: str
    body_style: str


    def __post_init__(self) -> None:
        if not all(value.strip() for value in (self.fuel_type, self.transmission, self.body_style)):
            raise ValueError("Характеристики транспортного засобу не можуть бути порожніми")



@dataclass(slots=True, frozen=True)
class Vehicle(ABC):
    """Базова сутність каталогу; конкретний транспорт має ціну та характеристики."""


    brand: str
    model: str
    year: int
```

```
    price: float
    mileage: int
    specification: VehicleSpecification


    def __post_init__(self) -> None:
        if not self.brand.strip() or not self.model.strip():
            raise ValueError("Марка та модель не можуть бути порожніми")
        if self.year < 1886:
            raise ValueError(f"Некоректний рік випуску: {self.year}")
        if not math.isfinite(self.price) or self.price < 0:
            raise ValueError("Ціна має бути скінченною та невід'ємною")
        if self.mileage < 0:
            raise ValueError("Пробіг не може бути від'ємним")


    @property
    def full_name(self) -> str:
        return f"{self.brand} {self.model}"


    def __str__(self) -> str:
        return f"{self.full_name} ({self.year}) — ${self.price:,.2f}"



@dataclass(slots=True, frozen=True)
class Car(Vehicle):
    """Модель даних автомобіля.


    Атрибути:
        brand: Марка автомобіля.
        model: Модель автомобіля.
        year: Рік випуску (>= 1886).
        price: Ціна в USD (>= 0).
        mileage: Пробіг у кілометрах (>= 0).
    """


    specification: VehicleSpecification = VehicleSpecification("Бензин", "Автоматична", "Седан")



@dataclass(slots=True, frozen=True)
class ElectricCar(Car):
    """Автомобіль з електричним приводом; запас ходу задається в кілометрах."""


    range_km: int = 0


    def __post_init__(self) -> None:
        super(ElectricCar, self).__post_init__()
        if self.range_km <= 0:
            raise ValueError("Запас ходу електромобіля має бути додатним")
        if self.specification.fuel_type.casefold() not in {"електро", "electric"}:
            raise ValueError("Для ElectricCar тип пального має бути 'Електро'")
```

criteria.py

Лістинг наведено з робочого файлу src/car_catalog/criteria.py; форматування спрощено для звіту.

```
"""Реалізації критеріїв пошуку каталогу."""


from dataclasses import dataclass
```

```
from car_catalog.models import Vehicle



@dataclass(frozen=True, slots=True)
class BrandModelCriteria:
    brand: str | None = None
    model: str | None = None


    def matches(self, item: Vehicle, /) -> bool:
        return (
            (self.brand is None or item.brand.casefold() == self.brand.strip().casefold())
            and (self.model is None or item.model.casefold() == self.model.strip().casefold())
        )



@dataclass(frozen=True, slots=True)
class YearCriteria:
    year: int


    def matches(self, item: Vehicle, /) -> bool:
        return item.year == self.year
```

protocols.py

Лістинг наведено з робочого файлу src/car_catalog/protocols.py; форматування спрощено для звіту.

```
"""Абстракції для пошуку та збереження каталогу."""


from collections.abc import Iterable
from typing import Protocol, TypeVar, runtime_checkable


from car_catalog.models import Vehicle


T_contra = TypeVar("T_contra", contravariant=True)



@runtime_checkable
class SearchCriteria(Protocol[T_contra]):
    """Structural interface критерію пошуку."""


    def matches(self, item: T_contra, /) -> bool: ...



class VehicleRepository(Protocol):
    """Мінімальний інтерфейс сховища, потрібний сервісу каталогу."""


    def add(self, item_id: int, item: Vehicle) -> None: ...


    def get(self, item_id: int) -> Vehicle | None: ...


    def all(self) -> Iterable[Vehicle]: ...
```

repositories.py

Лістинг наведено з робочого файлу src/car_catalog/repositories.py; форматування спрощено для звіту.

```
"""Універсальне in-memory сховище та конкретний репозиторій авто."""


from abc import ABC, abstractmethod
from collections.abc import Iterable, Iterator
from typing import Generic, TypeVar


from car_catalog.models import Vehicle


T = TypeVar("T")



class Repository(ABC, Generic[T]):
    """Контракт типізованого сховища сутностей."""


    @abstractmethod
    def add(self, item_id: int, item: T) -> None: ...


    @abstractmethod
    def get(self, item_id: int) -> T | None: ...


    @abstractmethod
    def all(self) -> Iterable[T]: ...



class InMemoryRepository(Repository[T]):
    """Generic repository з інкапсульованим індексом за ID."""


    def __init__(self) -> None:
        self._items: dict[int, T] = {}


    def add(self, item_id: int, item: T) -> None:
        if item_id <= 0:
            raise ValueError("ID має бути додатним")
        self._items[item_id] = item


    def get(self, item_id: int) -> T | None:
        return self._items.get(item_id)


    def all(self) -> Iterable[T]:
        return tuple(self._items.values())



class VehicleCatalog:
    """Колекція транспортних засобів, що композиційно використовує repository."""


    def __init__(self, repository: Repository[Vehicle] | None = None) -> None:
        self._repository: Repository[Vehicle] = repository or InMemoryRepository[Vehicle]()
        self._next_id = 1


    def add(self, vehicle: Vehicle) -> int:
        item_id = self._next_id
```

```
        self._repository.add(item_id, vehicle)
        self._next_id += 1
        return item_id


    def get(self, item_id: int) -> Vehicle | None:
        return self._repository.get(item_id)


    def all(self) -> tuple[Vehicle, ...]:
        return tuple(self._repository.all())


    def __len__(self) -> int:
        return sum(1 for _ in self._repository.all())


    def __iter__(self) -> Iterator[Vehicle]:
        return iter(self._repository.all())


    def __contains__(self, item_id: object) -> bool:
        return isinstance(item_id, int) and self._repository.get(item_id) is not None


    def __getitem__(self, item_id: int) -> Vehicle:
        vehicle = self._repository.get(item_id)
        if vehicle is None:
            raise KeyError(item_id)
        return vehicle
```

dto.py

Лістинг наведено з робочого файлу src/car_catalog/dto.py; форматування спрощено для звіту.

```
"""TypedDict контракт для JSON-подібного представлення авто."""


from typing import TypedDict



class VehicleData(TypedDict):
    id: int
    brand: str
    model: str
    year: int
    price: float
    mileage: int
    kind: str
```

application.py

Лістинг наведено з робочого файлу src/car_catalog/application.py; форматування спрощено для звіту.

```
"""Application service: use cases каталогу з DI та залежністю від Protocol."""


from car_catalog.criteria import BrandModelCriteria, YearCriteria
from car_catalog.dto import VehicleData
from car_catalog.models import ElectricCar, Vehicle
from car_catalog.protocols import SearchCriteria, VehicleRepository
```

```
class VehicleCatalogService:
    """Виконує сценарії каталогу, отримуючи repository ззовні."""


    def __init__(self, repository: VehicleRepository) -> None:
        self._repository = repository
        self._next_id = 1


    def add(self, vehicle: Vehicle) -> int:
        item_id = self._next_id
        self._repository.add(item_id, vehicle)
        self._next_id += 1
        return item_id


    def search(self, criteria: SearchCriteria[Vehicle]) -> list[Vehicle]:
        return [vehicle for vehicle in self._repository.all() if criteria.matches(vehicle)]


    def search_by_brand_or_model(self, brand: str | None = None, model: str | None = None) ->
list[Vehicle]:
        return self.search(BrandModelCriteria(brand=brand, model=model))


    def filter_by_year(self, year: int) -> list[Vehicle]:
        return self.search(YearCriteria(year))


    def most_expensive(self) -> Vehicle | None:
        return max(self._repository.all(), key=lambda vehicle: vehicle.price, default=None)


    def average_price(self) -> float:
        vehicles = tuple(self._repository.all())
        return sum(vehicle.price for vehicle in vehicles) / len(vehicles) if vehicles else 0.0


    def lowest_mileage(self) -> Vehicle | None:
        return min(self._repository.all(), key=lambda vehicle: vehicle.mileage, default=None)


    def to_data(self, item_id: int, vehicle: Vehicle) -> VehicleData:
        return {
            "id": item_id,
            "brand": vehicle.brand,
            "model": vehicle.model,
            "year": vehicle.year,
            "price": vehicle.price,
            "mileage": vehicle.mileage,
            "kind": "electric" if isinstance(vehicle, ElectricCar) else "car",
        }
```

oop_demo.py

Лістинг наведено з робочого файлу src/car_catalog/oop_demo.py; форматування спрощено для звіту.

```
"""Демонстрація об'єктної моделі ЛР4 для варіанта 5."""


from car_catalog.application import VehicleCatalogService
from car_catalog.models import Car, ElectricCar, Vehicle, VehicleSpecification
from car_catalog.repositories import InMemoryRepository




def run_demo() -> None:
    repository: InMemoryRepository[Vehicle] = InMemoryRepository()
    service = VehicleCatalogService(repository)
    service.add(Car("Toyota", "Camry", 2020, 25_000.0, 45_000))
```

```
    service.add(Car("BMW", "M3", 2022, 72_000.0, 18_000))
    service.add(
        ElectricCar(
            "Tesla", "Model 3", 2023, 41_000.0, 12_000,
            VehicleSpecification("Електро", "Автоматична", "Седан"), 513,
        )
    )


    print("\n--- ООП-модель каталогу (ЛР4, варіант 5) ---")
    print("Пошук Toyota:", [str(car) for car in service.search_by_brand_or_model(brand="Toyota")])
    print("Авто 2023 року:", [str(car) for car in service.filter_by_year(2023)])
    print("Найдорожче авто:", service.most_expensive())
    print(f"Середня ціна: ${service.average_price():,.2f}")
    print("Мінімальний пробіг:", service.lowest_mileage())



if __name__ == "__main__":
    run_demo()
```

19. Висновок

У лабораторній роботі №4 для варіанта 5 побудовано типізовану об’єктну модель каталогу автомобілів у межах наскрізного проєкту. Реалізовано Vehicle, Car та ElectricCar, frozen VehicleSpecification, composition каталогу з generic repository, ABC і Protocol-контракти, search criteria, TypedDict DTO, application service і демонстраційний сценарій із dependency injection.

Дизайн застосовує SRP, OCP, LSP, ISP і DIP: відповідальності розділено, пошук розширюється новими критеріями, підтип зберігає контракт базового автомобіля, інтерфейси невеликі, а сервіс залежить від abstraction. Статичний експеримент підтвердив, що mypy виявляє три задані помилки типів; після виправлення аналізатор повідомляє Success: no issues found in 7 source files.

Результатом є сумісне продовження проєкту ЛР1–ЛР3 із чіткою domain model та межами між моделлю, інфраструктурою і application layer.